

بخش اول

مقدمه

در این بخش به دنبال این هستیم که جداسازی منابع کور را با فرض استقلال منابع حل کنیم.

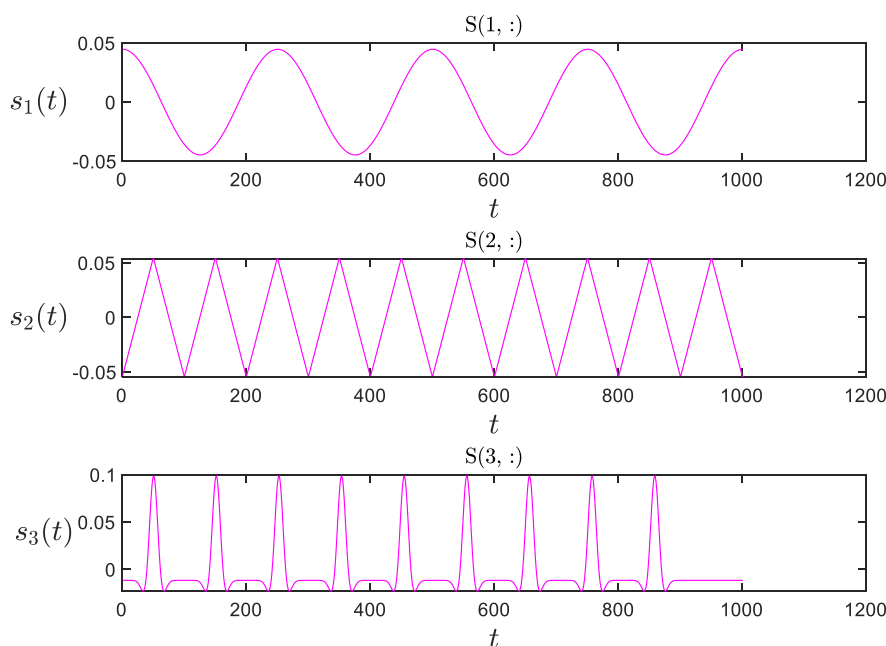
در قسمت ۰، ابتدا منابع و مشاهدات بدون نویز و مشاهدات نویزی را رسم می کنیم.

سپس در قسمت ۱، به دنبال این هستیم که ماتریس جدا کننده را بدست آوریم. در ادامه و در قسمت ۳، ابهام منابع و ابهام ترتیب را حذف می کنیم و خطا را بدست می آوریم.

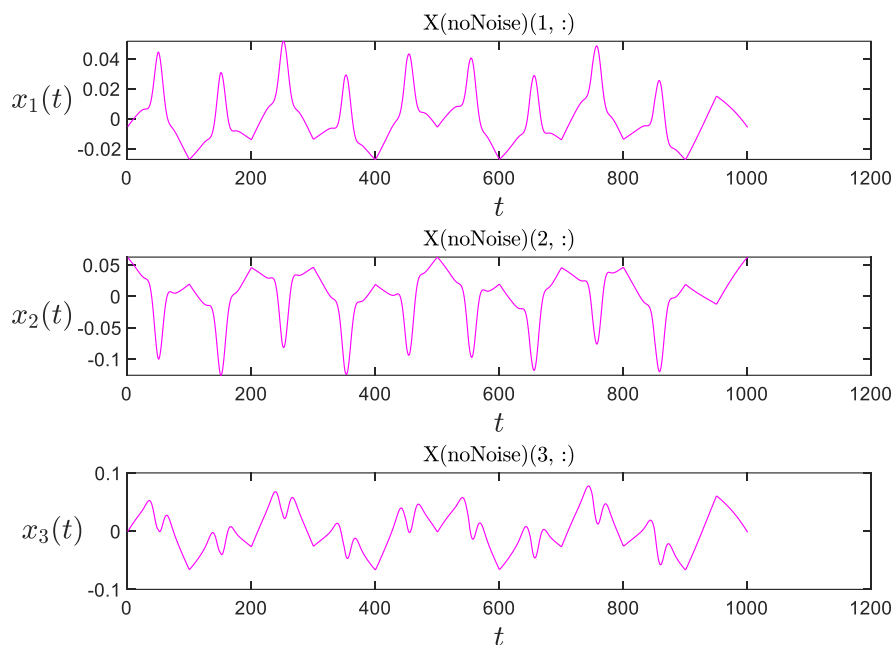
در قسمت ۳، نمودار همگرایی را رسم و نتایج بدست آمده را گزارش می کنیم.

قسمت ۰

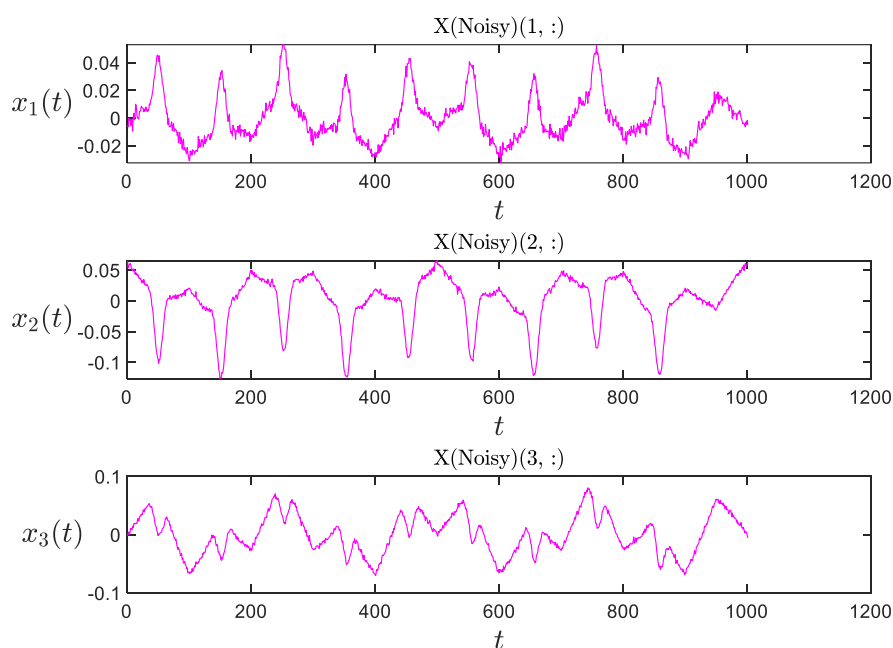
ابتدا از فرمول های $X = A S + Noise$ ، ماتریس مشاهدات نویزی را بدست می آوریم. همچنین، از فرمول $X = A S$ نیز مشاهدات بدون نویز بدست می آید. حال، نمودارهای خواسته شده را رسم می کنیم. تصویر ۱.۱ نمودار منابع، تصویر ۱.۲ نمودار مشاهدات بدون نویز و تصویر ۱.۳ نمودار مشاهدات نویزی را نشان می دهد.



تصویر ۱.۱: منابع



تصویر ۱.۲: مشاهدات بدون نویز



تصویر ۱.۳: مشاهدات نویزی

قسمت ۱

در این قسمت به دنبال پیاده سازی روش **deflation ICA** می باشیم. همچنین، برای تخمین تابع از روش **MSE** استفاده می کنیم. حال طبق الگوریتم های توضیح داده شده در کلاس برای روش **deflation ICA** داریم:

$$f(B) = \sum_{m=1}^M H(y_m) \rightarrow \begin{cases} B^{(k+1)} \leftarrow B^{(k)} - \mu \frac{\partial f}{\partial B} \\ \text{normalizing rows of } B^{(k+1)} \end{cases}$$

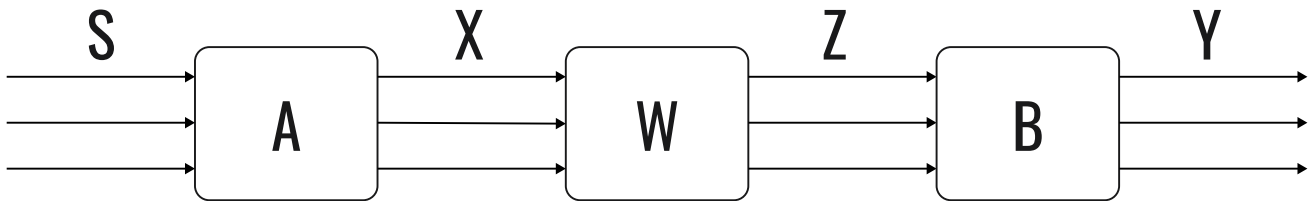
$$\text{Score Function: } \psi_Y(y) = -\frac{P_Y'(y)}{P_Y(y)}$$

$$MSE: \hat{\psi}_Y(y) = \theta^T K(y) = \theta^T \begin{bmatrix} 1 \\ y \\ y^2 \\ y^3 \\ y^4 \\ y^5 \end{bmatrix} \rightarrow \begin{cases} g(\theta) = E\{\theta^T K(y) K^T(y) \theta\} - 2E\{\theta^T K'(y)\} \\ \frac{\partial g}{\partial \theta} = 2E\{K(y) K^T(y)\} - 2E\{K'(y)\} = 0 \end{cases}$$

$$\rightarrow \hat{\theta} = (E\{K(y) K^T(y)\})^{-1} E\{K'(y)\} \Rightarrow \hat{\theta} : \checkmark \rightarrow \hat{\psi}_Y(y) : \checkmark \rightarrow \hat{\psi}_Y(y) = \hat{\theta} K(y)$$

$$\rightarrow \frac{\partial f}{\partial B} = (E\{\psi_Y(y) z^T\})^T$$

تصویر ۱.۴، نمودار کلی مسئله BSS در این الگوریتم را نشان می دهد.



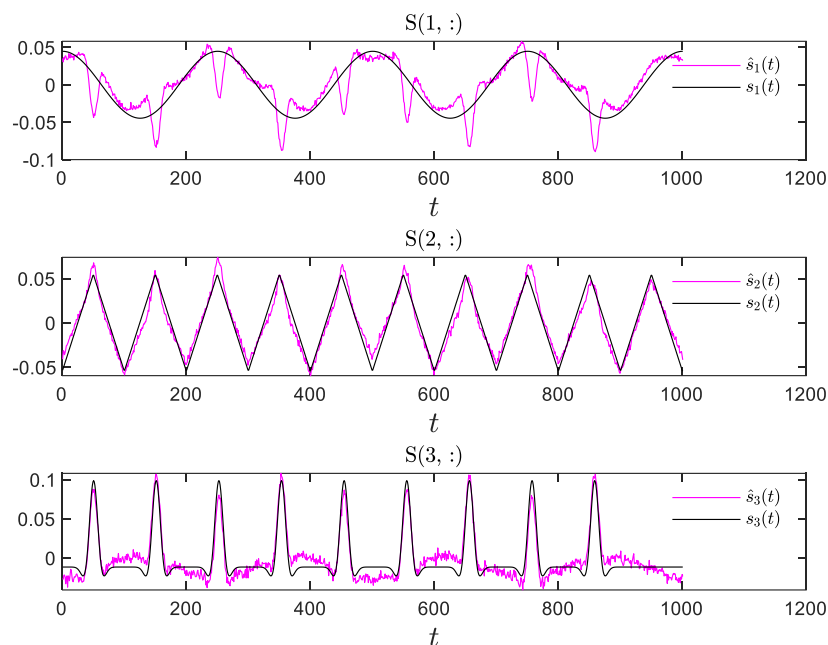
تصویر ۱.۴: نمودار کلی مسئله BSS برای ICA deflation

حال، ماتریس بدست آمده از ضرب ماتریس جدا کننده و ماتریس مخلوط کننده اصلی را بدست می آوریم. این ماتریس از رابطه $permutation_matrix = B \times (\Lambda^{-\frac{1}{2}} U^T) \times A$ بدست می آید. این ماتریس به صورت زیر می باشد:

$$permutation_matrix = \begin{bmatrix} -0.0296 & 0.1216 & 0.9134 \\ -0.9901 & -0.0142 & -0.0159 \\ -0.0185 & -1.1489 & 0.6844 \end{bmatrix}$$

همان طور که مشاهده می شود درایه نشان داده شده باعث شده تا ماتریس بدست آمده از permutation matrix دور باشد.

تصویر ۱.۵ مقایسه منابع تخمین زده شده از مشاهدات نویزی با منابع اصلی را نشان می دهد.



تصویر ۱.۵: مقایسه منابع تخمین زده شده از مشاهدات نویزی با منابع اصلی

قسمت ۲

رابطه خطا از فرمول زیر بدست می آید:

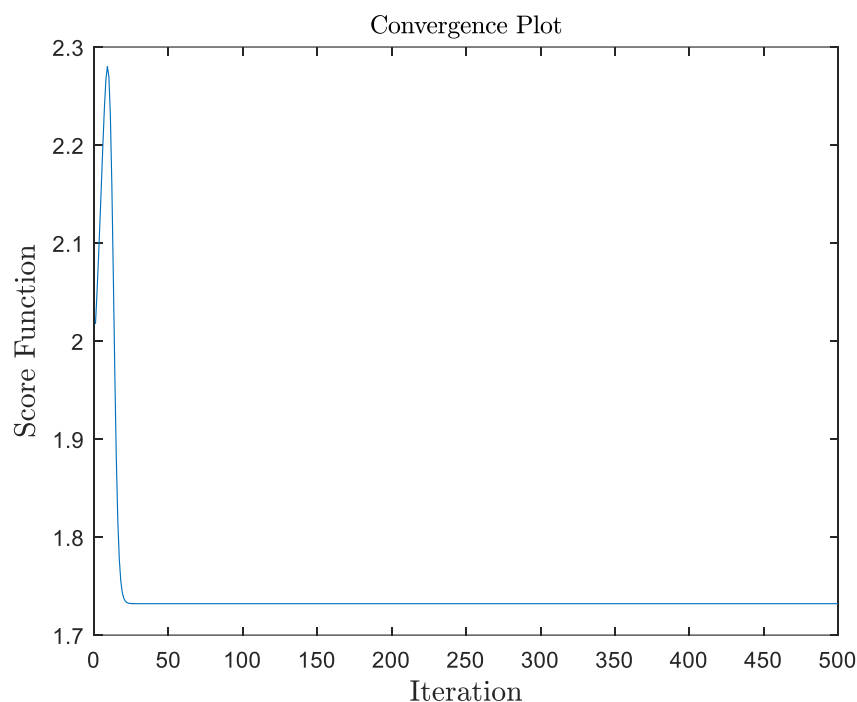
$$E = \frac{\|\hat{S} - S\|_F^2}{\|S\|_F^2}$$

نتایج بدست آمده از یک بار اجرای کد به صورت زیر می باشد:

$$E = 0.17911$$

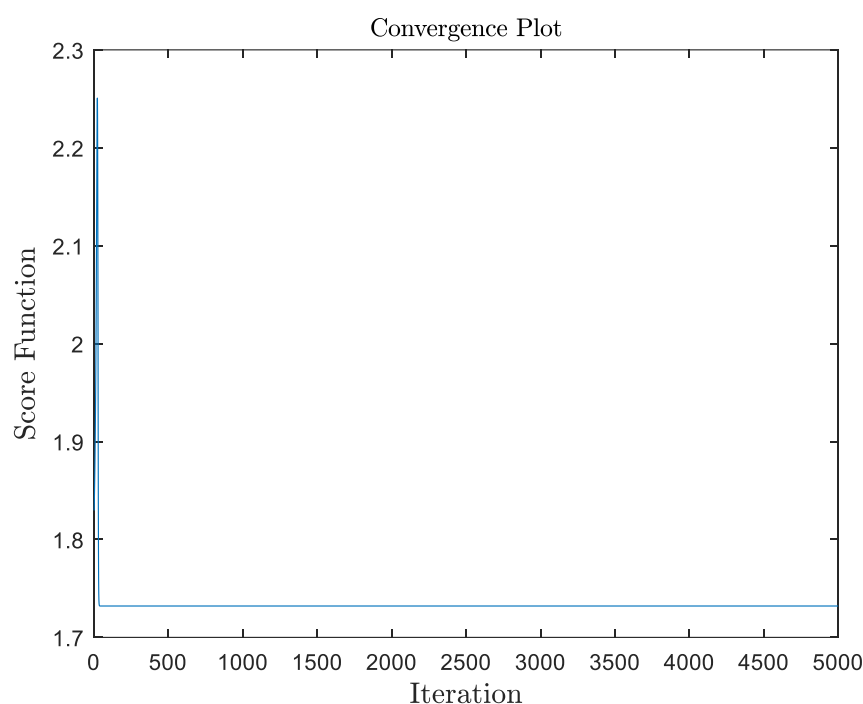
قسمت ۳

اگر چه تابع هدف برابر با $f(B) = \sum_{m=1}^M H(y_m)$ می باشد، اما $\frac{\partial f}{\partial B}$ را به عنوان تابع هدف در نظر می گیریم و نمودار آن را بر حسب شماره iteration رسم می کنیم. نمودار این تابع هدف در تصویر ۱.۶ آمده است.



تصویر ۱.۶: نمودار تابع هدف بر حسب شماره *iteration* (500 iteration)

برای مشاهده بهتر این تابع هدف، تعداد *iteration* ها را بیشتر می کنیم (10 برابر می کنیم) و مجدداً این نمودار را رسم می کنیم. تصویر ۱.۷ این نمودار را نشان می دهد.



تصویر ۱.۷: نمودار تابع هدف بر حسب شماره *iteration* (5000 iteration)

همان طور که مشاهده می شود، تابع هدف مورد نظر طی *iteration* ها به 0 میل نمی کند و مقدار آن از یک *iteration* به بعد، ثابت می شود.

بخش دوم

قسمت ۱

در این قسمت به دنبال پیاده سازی روش **Equivariant** می باشیم. همچنین، برای تخمین تابع از روش **MSE** استفاده می کنیم. حال طبق الگوریتم های توضیح داده شده در کلاس برای روش **Equivariant** داریم:

$$f(B) = -H(y) + \sum_{m=1}^M H(y_m) \rightarrow \begin{cases} B^{(k+1)} \leftarrow B^{(k)} - \mu \frac{\partial f}{\partial B} \\ \text{normalizing rows of } B^{(k+1)} \end{cases}$$

البته مشاهده می شود که با شرایطی که در آینده اعمال خواهیم کرد، دیگر نیاز به قید نرمالیزه کردن نخواهیم داشت.

$$\text{Score Function: } \psi_Y(y) = -\frac{P_Y'(y)}{P_Y(y)}$$

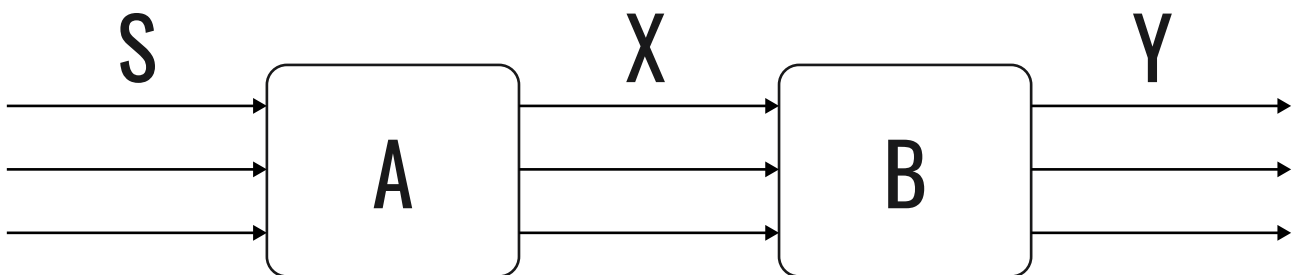
$$MSE: \hat{\psi}_Y(y) = \theta^T K(y) = \theta^T \begin{bmatrix} 1 \\ y \\ y^2 \\ y^3 \\ y^4 \\ y^5 \end{bmatrix} \rightarrow \begin{cases} g(\theta) = E\{\theta^T K(y) K^T(y) \theta\} - 2E\{\theta^T K'(y)\} \\ \frac{\partial g}{\partial \theta} = 2E\{K(y) K^T(y)\} - 2E\{K'(y)\} = 0 \end{cases}$$

$$\rightarrow \hat{\theta} = (E\{K(y) K^T(y)\})^{-1} E\{K'(y)\} \Rightarrow \hat{\theta} : \checkmark \rightarrow \hat{\psi}_Y(y) : \checkmark \rightarrow \hat{\psi}_Y(y) = \hat{\theta} K(y)$$

$$\rightarrow \frac{\partial f}{\partial B} = D \times B$$

$$D = \begin{cases} D_{i,i} = 1 - E\{y_i^2\} \\ D_{i,j} = E\{\psi_{Y_i}(y_i) y_j\} \end{cases}$$

تصویر ۲.۱، نمودار کلی مسئله **BSS** در این الگوریتم را نشان می دهد.



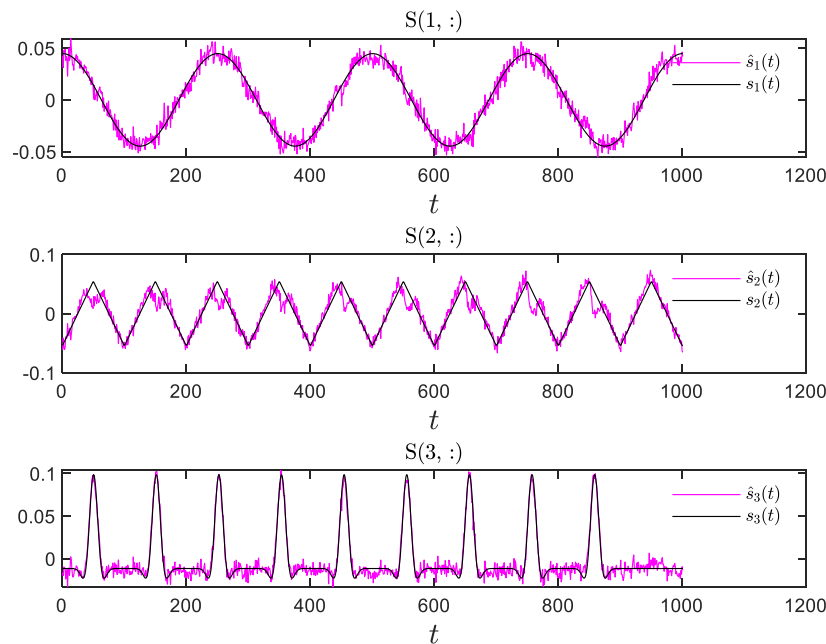
تصویر ۲.۱: نمودار کلی مسئله **BSS** برای **Equivariant**

حال، ماتریس بدست آمده از ضرب ماتریس جدا کننده و ماتریس مخلوط کننده اصلی را بدست می آوریم. این ماتریس از رابطه $permutation_matrix = B \times A$ بدست می آید. این ماتریس به صورت زیر می باشد:

$$\text{permutation_matrix} = \begin{bmatrix} 0.0134 & -0.0667 & -0.4679 \\ -0.4374 & -0.0420 & 0.0426 \\ 0.0127 & -0.3192 & 0.2220 \end{bmatrix}$$

همان طور که مشاهده می شود درایه نشان داده شده باعث شده تا ماتریس بدست آمده از permutation matrix دور باشد.

تصویر ۲.۲ مقایسه منابع تخمین زده شده از مشاهدات نویزی با منابع اصلی را نشان می دهد.



تصویر ۲.۲: مقایسه منابع تخمین زده شده از مشاهدات نویزی با منابع اصلی

قسمت ۲

رابطه خطا از فرمول زیر بدست می آید:

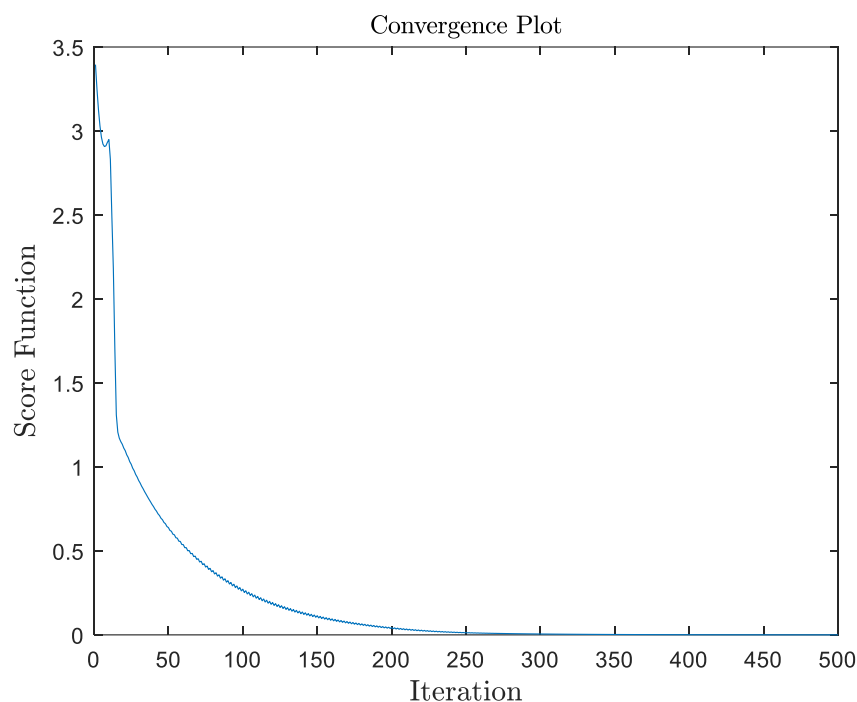
$$E = \frac{\|\hat{S} - S\|_F^2}{\|S\|_F^2}$$

نتایج بدست آمده از یک بار اجرای کد به صورت زیر می باشد:

$$E = 0.08237$$

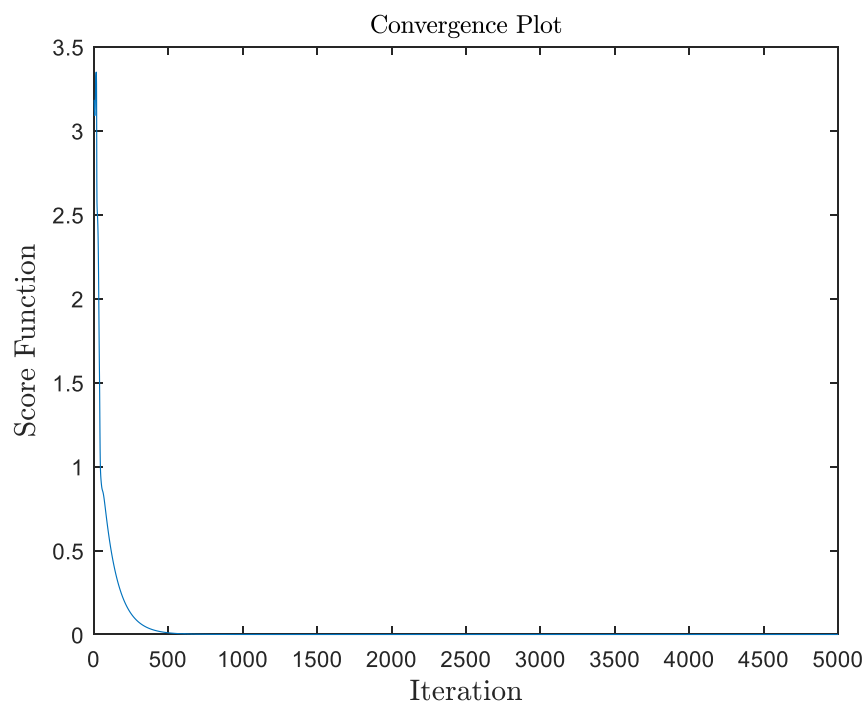
قسمت ۳

اگر چه تابع هدف برابر با $f(B) = -H(y) + \sum_{m=1}^M H(y_m)$ می باشد، اما $\frac{\partial f}{\partial B}$ را به عنوان تابع هدف در نظر می گیریم و نمودار آن را بر حسب شماره iteration رسم می کنیم. نمودار این تابع هدف در تصویر ۲.۳ آمده است.



تصویر ۲.۳: نمودار تابع هدف بر حسب شماره *iteration* (500 iteration)

برای مشاهده بهتر این تابع هدف، تعداد *iteration* ها را بیشتر می کنیم (10 برابر می کنیم) و مجدداً این نمودار را رسم می کنیم. تصویر ۲.۴ این نمودار را نشان می دهد.



تصویر ۲.۴: نمودار تابع هدف بر حسب شماره *iteration* (5000 iteration)

همان طور که مشاهده می شود، تابع هدف مورد نظر طی *iteration* ها به 0 میل می کند.

بخش سوم

بررسی سه الگوریتم به صورت مقایسه‌ای

اگر از نظر کیفیت جداسازی به بررسی روش‌ها بپردازیم، مشاهده می‌شود که روش Equivariant از سایر روش‌ها بهتر است زیرا پارامتر E آن به مقدار قابل توجهی کمتر از دو روش دیگر می‌باشد. همچنین اگر از مدت tic-toc زمان الگوریتم‌ها را بررسی کنیم به نتایج زیر می‌رسیم:

• زمان یک بار اجرای الگوریتم کمینه سازی D_{KL} : $1.089326 [s]$

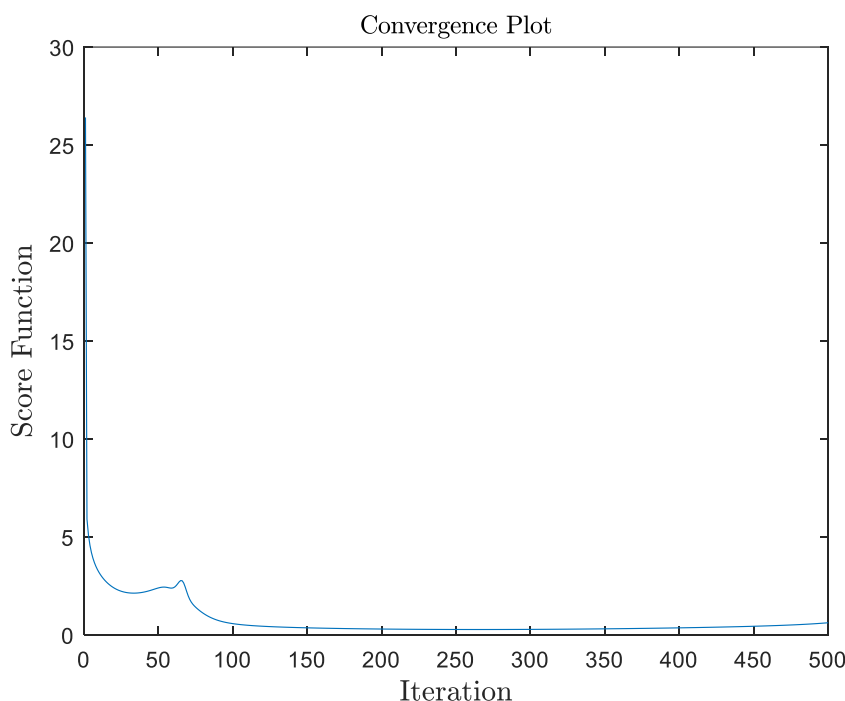
• زمان یک بار اجرای الگوریتم deflation ICA : $0.818347 [s]$

• زمان یک بار اجرای الگوریتم Equivariant : $0.918447 [s]$

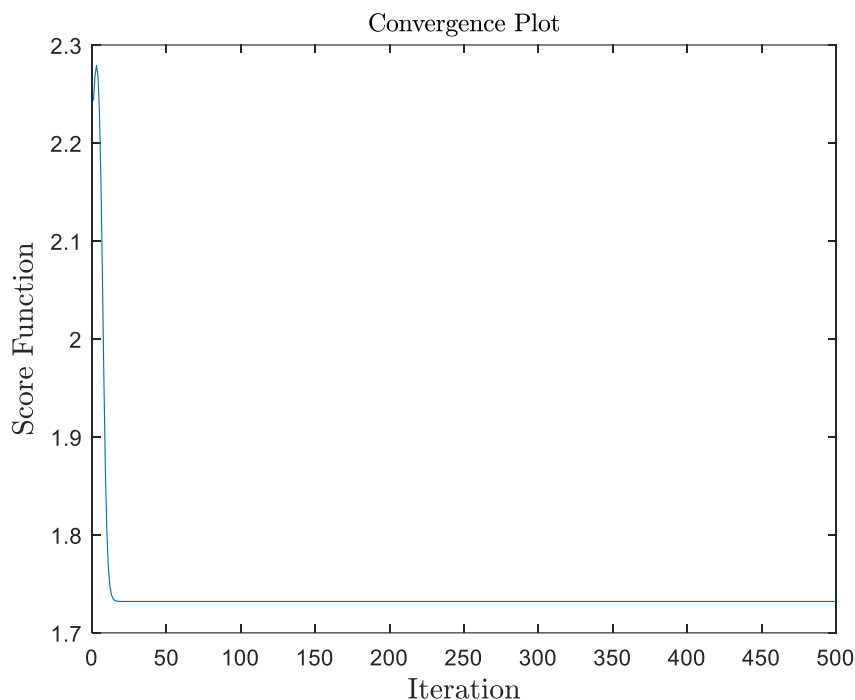
همان‌طور که مشاهده می‌شود، زمان اجرای این سه الگوریتم تقریباً مشابه یک دیگر می‌باشد؛ با این حال، اگر بخواهیم این سه الگوریتم را بر اساس زمان اجرا مرتب کنیم، داریم:

$$\text{deflation ICA} < \text{Equivariant} < \text{minimizing } D_{KL}$$

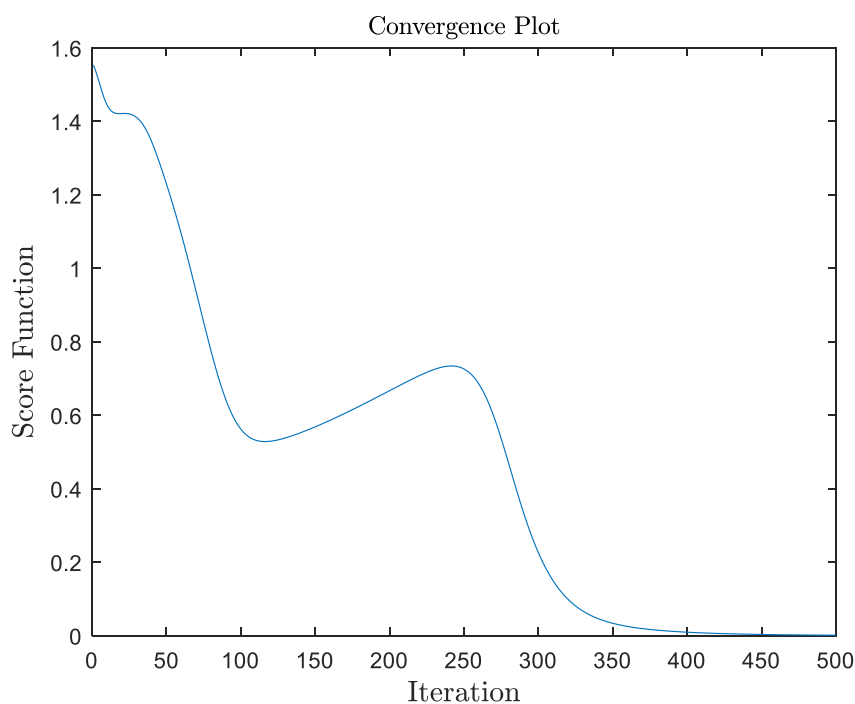
حال، نمودار همگرایی این سه الگوریتم را بررسی می‌کنیم. نمودار همگرایی الگوریتم کمینه سازی D_{KL} در تصویر ۳.۱، الگوریتم deflation ICA در تصویر ۳.۲، و الگوریتم Equivariant در تصویر ۳.۳ آمده است.



تصویر ۳.۱: همگرایی الگوریتم کمینه سازی D_{KL}



تصویر ۳.۲: همگرایی الگوریتم deflation ICA



تصویر ۳.۳: همگرایی الگوریتم Equivariant

مشاهده می شود که الگوریتم deflation ICA سرعت همگرایی بیشتری دارد، اما به صفر میل نمی کند. الگوریتم کمینه سازی D_{KL} نیز زود همگرا می شود. اما الگوریتم Equivariant با طی کردن iteration های بیشتری همگرا می شود. در نتیجه در مقایسه سرعت همگرایی، الگوریتم deflation ICA سرعت همگرایی بیشتری دارد.

بررسی سه الگوریتم به صورت تجربی

الگوریتم کمینه سازی D_{KL} : در این الگوریتم برای کمینه سازی تابع هدف، در هر $iteration$ باید تابع هزینه (cost function) را بیابیم. برای یافتن آن، باید آن را تخمین بزنیم. بعد از تخمین تابع هزینه، تابع هدف را می یابیم و در ادامه با روش Gradient Projection، به کمینه سازی D_{KL} می پردازیم. بدیهی است که این روش بسیار زمان بر و با خطای نسبتاً زیادی می باشد.

الگوریتم deflation ICA : در این الگوریتم، مجدداً باید در هر $iteration$ تابع هزینه را بیابیم، اما تفاوت این روش با روش بالا این است که در روش deflation ICA، به صورت مستقل به یافتن هر سطر ماتریس B می پردازیم. همچنین، به علت سفید سازی مشاهدات خود، ماتریس B ، orthonormal می باشد. در این روش، تابع هدف ما عبارت B^{-T} را ندارد. این روش، به دلیل یافتن هر سطر B به صورت جداگانه، از سرعت اجرا و سرعت همگرایی بهتری برخوردار است.

الگوریتم Equivariant : در این الگوریتم نیز مجدداً باید در هر $iteration$ تابع هزینه را بیابیم. در این الگوریتم، بلوک A و B را مجموعاً به صورت یک بلوک C فرض می کنیم و با بسط تیلور، به یافتن تابع هدف می پردازیم. این روش از خطای بسیار خوبی برخوردار است اما تعداد $iteration$ های زیادی را برای همگرایی تابع هدف خود نیاز دارد.

نتیجه

به صورت کلی، روش های deflation ICA و Equivariant الگوریتم های مناسبی می باشد. روش deflation ICA از زمان همگرایی بهتری برخوردار است؛ روش Equivariant کیفیت جداسازی بهتری دارد. به طور کلی روش Equivariant، روش مناسب تری می باشد.